

Parameter Estimation

Data Science

Parameter Estimation

- least squares
- maximum likelihood method
- maximum a posteriori
- confidence intervals

Statistical Inference

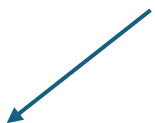
two main goals:

- **parameter estimation:** estimate unknown parameters from data
- **hypothesis testing:** make decisions about assumptions/models (discussed later in context of model evaluation)

What is Parameter Estimation?

assumption:

observed data $\{\mathbf{x}_i\}_{i=1}^n$ sampled from a probability distribution of random variables $\mathbf{X}$



$$p_{\mathbf{X}}(\mathbf{x}|\boldsymbol{\theta})$$

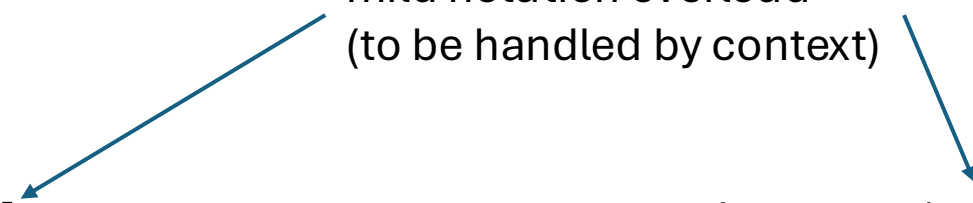
→ model

with unknown parameters $\boldsymbol{\theta}$

goal: estimate $\boldsymbol{\theta}$ from observed data

Notation

mild notation overload
(to be handled by context)



- random variable: X
- vector of several random variables: $\mathbf{X}$

- observation of random variable X : x
- observation of random vector $\mathbf{X}$: $\mathbf{x}$

- column vector: $\mathbf{x}$
- row vector: $\mathbf{x}^T$

features



- multiple observations
 - of X : $x_1, \dots, x_n$ or $\{x_i\}_{i=1}^n$
 - of $\mathbf{X}$: $\mathbf{x}_1, \dots, \mathbf{x}_n$ or $\{\mathbf{x}_i\}_{i=1}^n$

samples



- data matrix X
 - rows: samples
 - columns: features

- parameter: θ
- vector of parameters $\boldsymbol{\theta}$

- probability that X takes on value x : $p(x)$

probability distribution
abbreviation for $p_X(x)$



Two Types of Parameter Estimation Tasks

unsupervised setup (no labels)

- data: $\mathbf{x}_1, \dots, \mathbf{x}_n$
- model: $p(\mathbf{x}; \boldsymbol{\theta})$
- goal: fit distribution of data

ML perspective:

unsupervised learning
(e.g., GMM in clustering)

supervised setup (with labels)

- data: $(\mathbf{x}_i, y_i)$ pairs
- model: $p(y|\mathbf{x}; \boldsymbol{\theta})$
- goal: predict y from $\mathbf{x}$

supervised learning

Unsupervised Setup: Examples

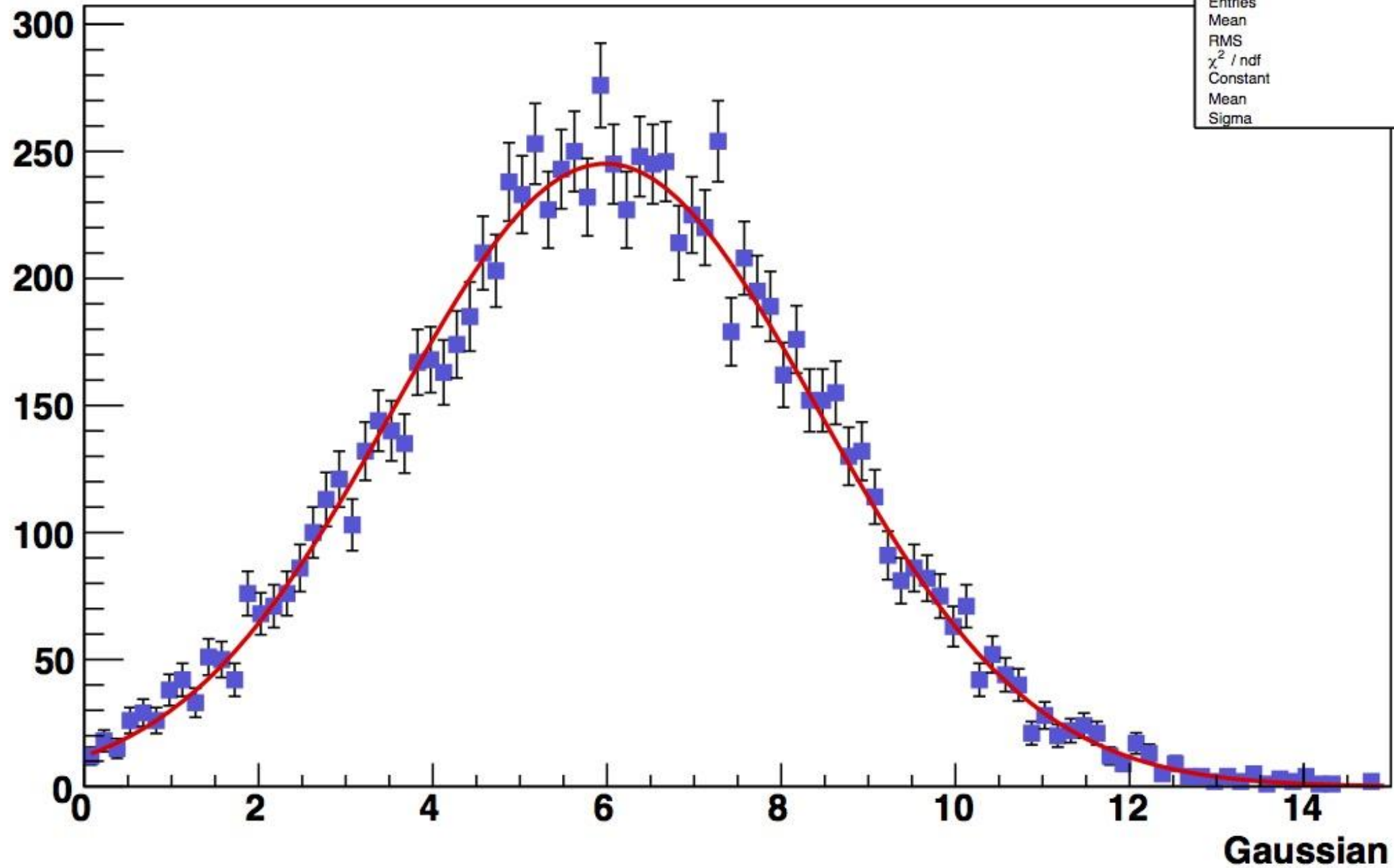
mean of a distribution of $X \sim p(x; \boldsymbol{\theta})$

- parameter of interest: $\mu = \mathbb{E}[X]$ (think of μ as θ_1)
- estimated from observed data $x_1, \dots, x_n$: $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$
- examples: mean of Gaussian, probability of success in a Bernoulli

variance of a distribution (e.g., variance of Gaussian)

- parameter of interest: $\sigma^2 = \text{Var}[X]$ (think of σ^2 as θ_2)
- estimated from sample variance

Function to be fit

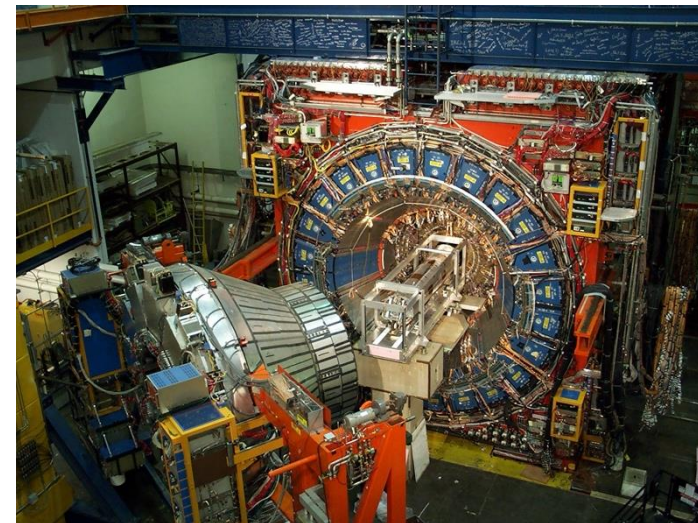
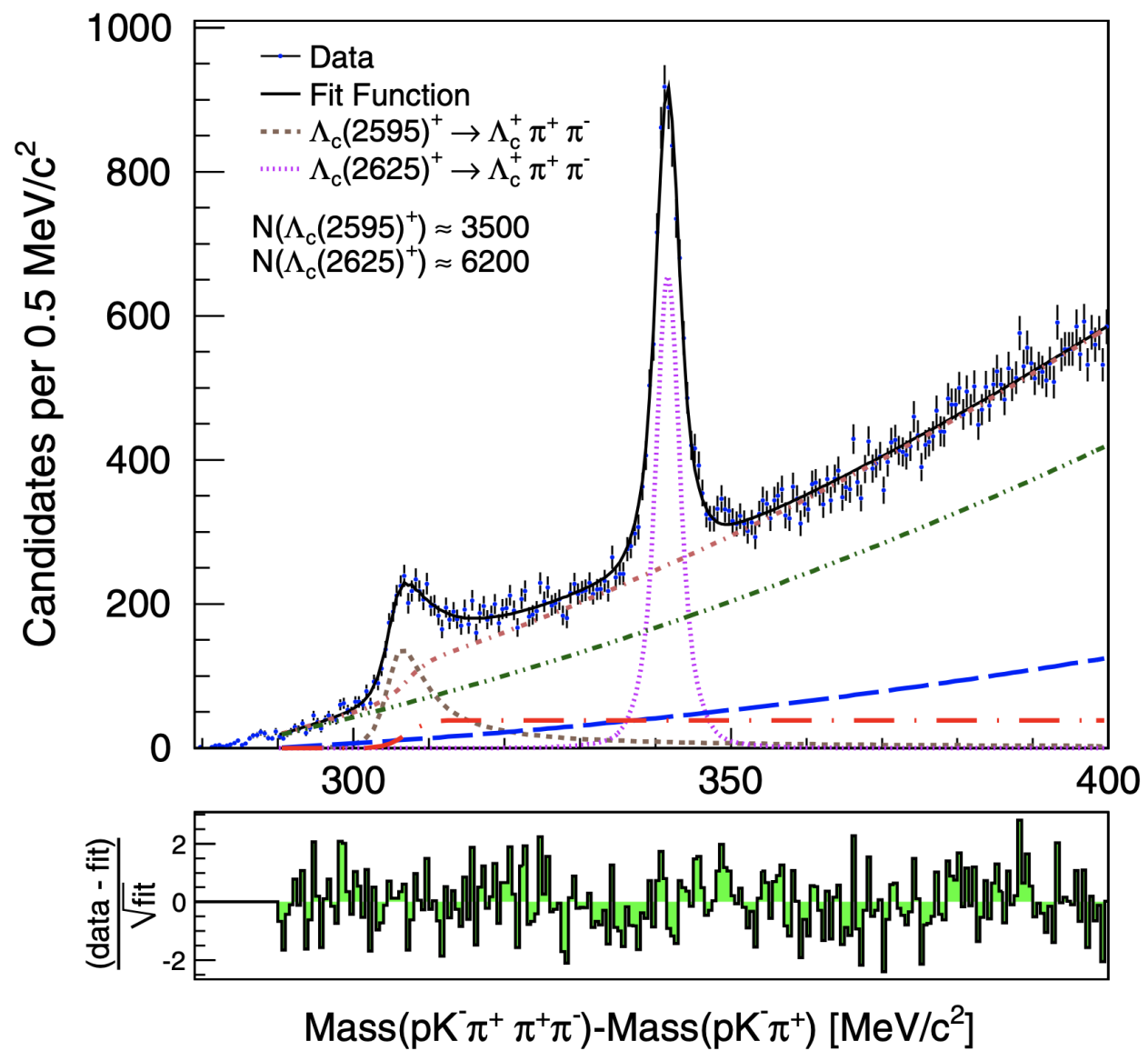


Unsupervised Setup: General Principle

estimate any θ of assumed $p(x; \theta)$

find parameter values such that the assumed model best explains the observed data

→ typically requires solving optimization problem (e.g. least squares, maximum likelihood)



Supervised Setup

$$\mathbf{X} \sim p(\mathbf{x}), Y|\mathbf{X} = \mathbf{x} \sim p(y|\mathbf{x}; \boldsymbol{\theta})$$

→ sampling conditional $(\mathbf{x}, y)$ pairs

goal: estimate $\boldsymbol{\theta}$ from observed data $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)$ by fitting $f(\mathbf{x}_i; \boldsymbol{\theta})$ to y_i

→ later predict $\hat{y}$ for new $\mathbf{x}$ from learned $f(\mathbf{x}; \hat{\boldsymbol{\theta}})$

- regression parameters (continuous outputs $y \in \mathbb{R}$)
- classification parameters (discrete outputs $y \in \{1, \dots, K\}$)

Example: Linear Regression

model (for each observation i):

$$y_i = \mathbf{w} \cdot \mathbf{x}_i + b + \varepsilon_i$$

$f(\mathbf{x}_i; \mathbf{w}, b)$ predictive parameters $\mathbf{w}, b$
(σ^2 nuisance parameter)

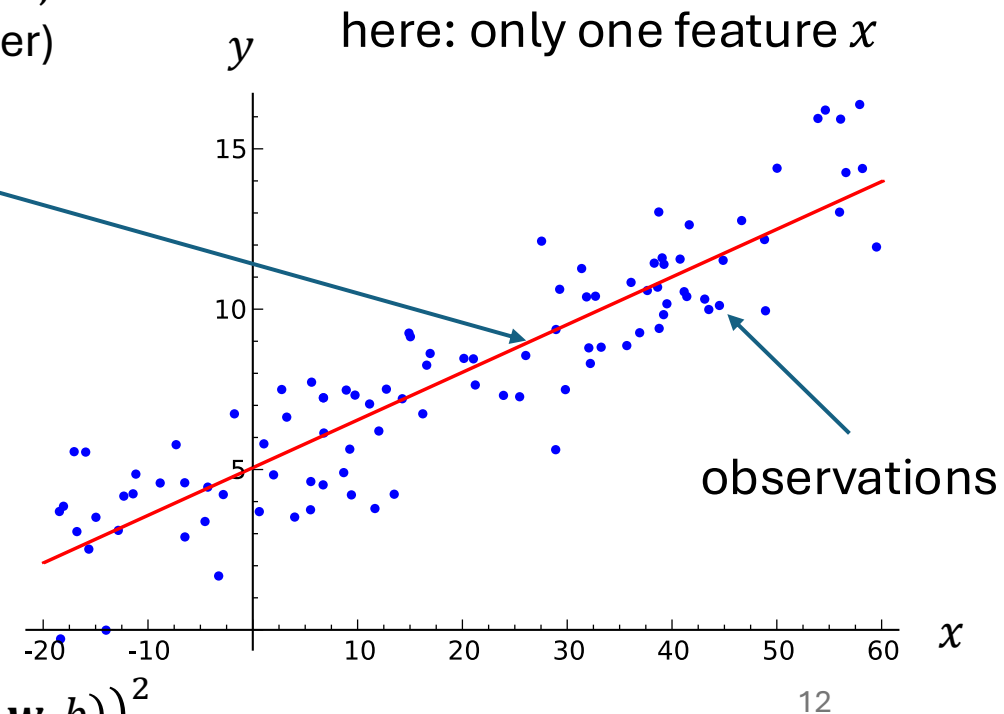
assumption:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\Rightarrow p(y|\mathbf{x}; \mathbf{w}, b, \sigma^2) = \mathcal{N}(\mathbf{w} \cdot \mathbf{x} + b, \sigma^2)$$

different $p(y|\mathbf{x})$ for each $\mathbf{x}$,
sharing parameters $\underbrace{\mathbf{w}, b, \sigma^2}_{\boldsymbol{\theta}}$

typically several variables $\mathbf{X}$
 $\rightarrow$ also several parameters $\mathbf{w}$



$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (y_i - f(\mathbf{x}_i; \mathbf{w}, b))^2$$

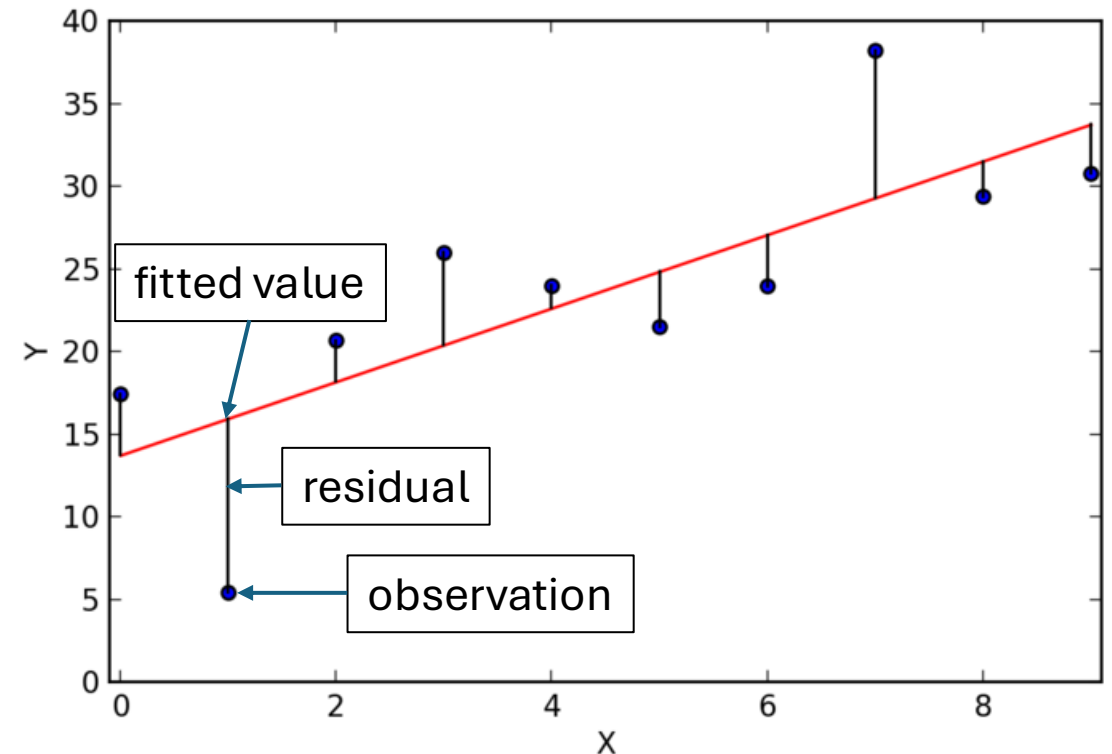
Least Squares Estimation

idea: choose parameters that minimize sum of squared residuals r_i

$$\operatorname{argmin}_{\boldsymbol{\theta}} \sum_{i=1}^n r_i(\boldsymbol{\theta})^2$$

examples for residuals:

- mean estimation: $r_i(\mu) = x_i - \mu$
- regression: $r_i(\boldsymbol{\theta}) = y_i - f(x_i; \boldsymbol{\theta})$



Least Squares Estimation: Minimization

minimization of objective

$$J(\boldsymbol{\theta}) = \sum_{i=1}^n r_i(\boldsymbol{\theta})^2$$

candidate solution: need
to ensure it is a minimum

first-order optimality condition: set gradient of objective to zero

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \sum_{i=1}^n \underbrace{2r_i(\boldsymbol{\theta}) \nabla_{\boldsymbol{\theta}} r_i(\boldsymbol{\theta})}_{\text{residuals appear multiplied by their sensitivity}} = 0$$

residuals appear multiplied by their sensitivity

→ $\hat{\boldsymbol{\theta}}$ satisfies $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = 0$

$$\nabla_{\boldsymbol{\theta}} = \begin{bmatrix} \frac{\partial}{\partial \theta_1} \\ \frac{\partial}{\partial \theta_2} \\ \vdots \end{bmatrix}$$

Linear Least Squares

model (linear in parameters):

$$r_i(\boldsymbol{\theta}) = y_i - \mathbf{x}_i^T \boldsymbol{\theta}$$

objective: $J(\boldsymbol{\theta}) = \sum_{i=1}^n (y_i - \mathbf{x}_i^T \boldsymbol{\theta})^2$

matrix form: $J(\boldsymbol{\theta}) = \|\mathbf{y} - X\boldsymbol{\theta}\|^2$

new notation: $\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \quad X = \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_n^T \end{bmatrix}$

linear regression:

$$r_i(\mathbf{w}, b) = y_i - (\mathbf{w} \cdot \mathbf{x}_i + b)$$

define $\boldsymbol{\theta} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$ and $\tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$

$$\rightarrow r_i(\boldsymbol{\theta}) = y_i - \tilde{\mathbf{x}}_i^T \boldsymbol{\theta}$$

Linear Least Squares: Solution

$$J(\boldsymbol{\theta}) = \|\mathbf{y} - X\boldsymbol{\theta}\|^2 = (\mathbf{y} - X\boldsymbol{\theta})^T (\mathbf{y} - X\boldsymbol{\theta}) = \mathbf{y}^T \mathbf{y} - \underbrace{\mathbf{y}^T X\boldsymbol{\theta} - \boldsymbol{\theta}^T X^T \mathbf{y}}_{\mathbf{y}^T X\boldsymbol{\theta} = (\boldsymbol{\theta}^T X^T \mathbf{y})^T = \boldsymbol{\theta}^T X^T \mathbf{y}} + \boldsymbol{\theta}^T X^T X \boldsymbol{\theta} \\ = \mathbf{y}^T \mathbf{y} - 2\boldsymbol{\theta}^T X^T \mathbf{y} + \boldsymbol{\theta}^T X^T X \boldsymbol{\theta}$$

$$\mathbf{y}^T X\boldsymbol{\theta} = (\boldsymbol{\theta}^T X^T \mathbf{y})^T = \boldsymbol{\theta}^T X^T \mathbf{y}$$

take gradient w.r.t. $\boldsymbol{\theta}$:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = -2X^T \mathbf{y} + 2X^T X \boldsymbol{\theta} = 2X^T (X\boldsymbol{\theta} - \mathbf{y})$$

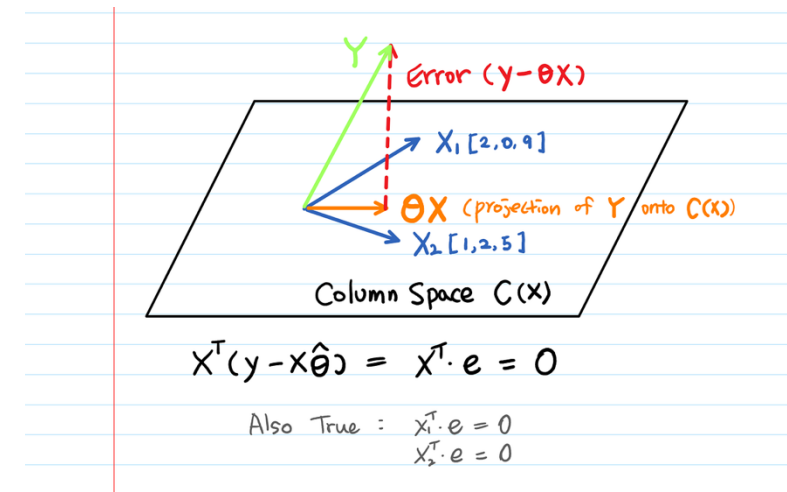
$$\nabla_{\boldsymbol{\theta}} (\boldsymbol{\theta}^T A \boldsymbol{\theta}) = (A + A^T) \boldsymbol{\theta} \\ \text{here: } A = X^T X, \text{ which is symmetric} \\ \rightarrow \nabla_{\boldsymbol{\theta}} (\boldsymbol{\theta}^T X^T X \boldsymbol{\theta}) = 2X^T X \boldsymbol{\theta}$$

set gradient to zero (optimality condition):

$$X^T X \boldsymbol{\theta} = X^T \mathbf{y}$$

solution (normal equations):

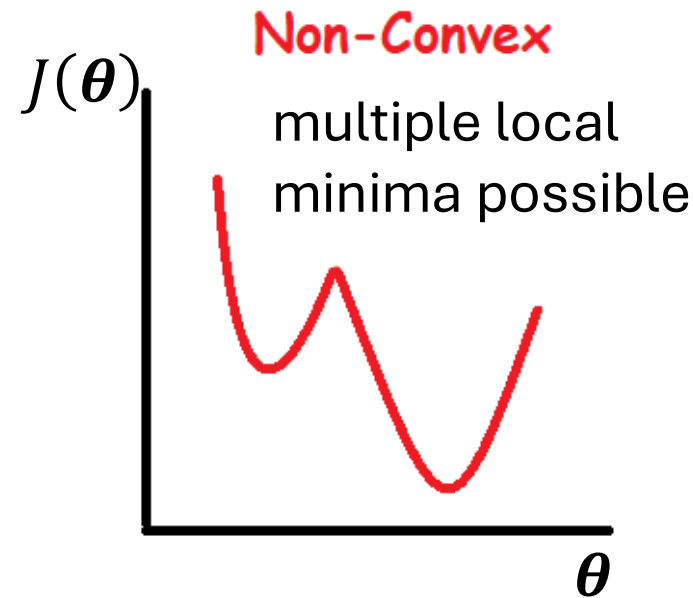
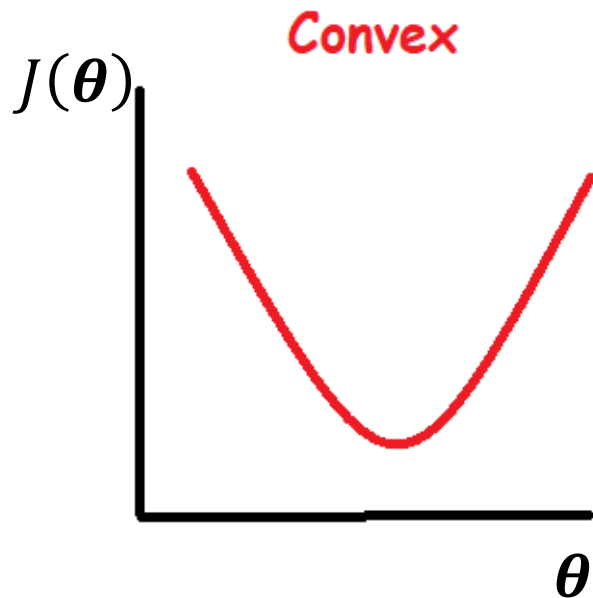
$$\hat{\boldsymbol{\theta}} = (X^T X)^{-1} X^T \mathbf{y}$$



General Nonlinear Case

$\nabla_{\theta} J(\theta) = 0$ gives stationary points

For convex problems (like linear least squares), this is the global minimum.



if $f(x; \theta)$ nonlinear in θ :
typically no closed-form
solution to $\nabla_{\theta} J(\theta) = 0$

→ iterative numerical
optimization

Gradient Descent

iteratively moving in direction of steepest descent (negative gradient):

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

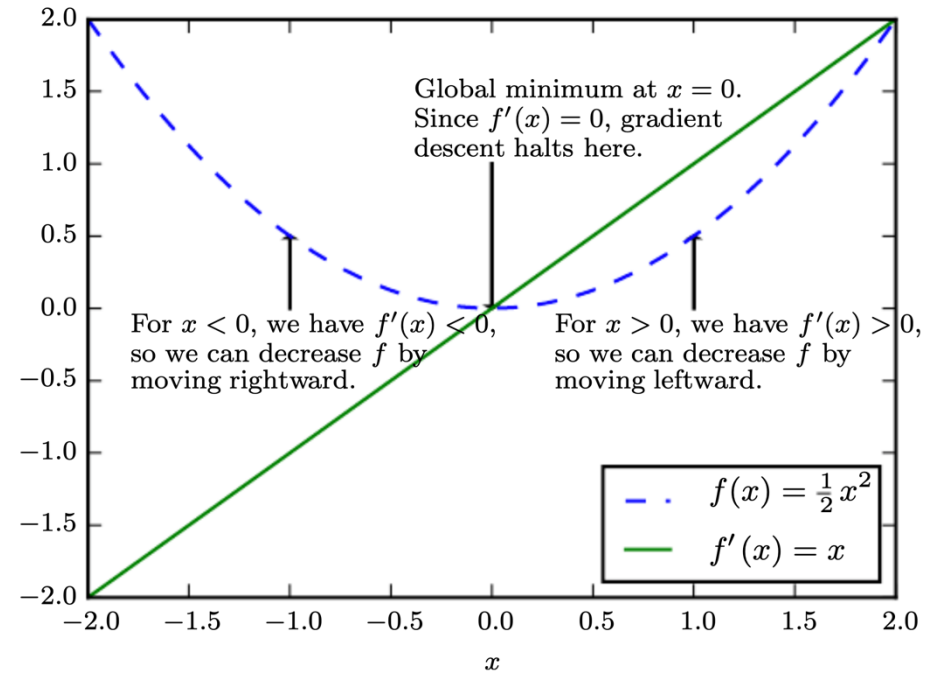
first-order

step size (learning rate): hyperparameter
controlling how aggressively to move downhill

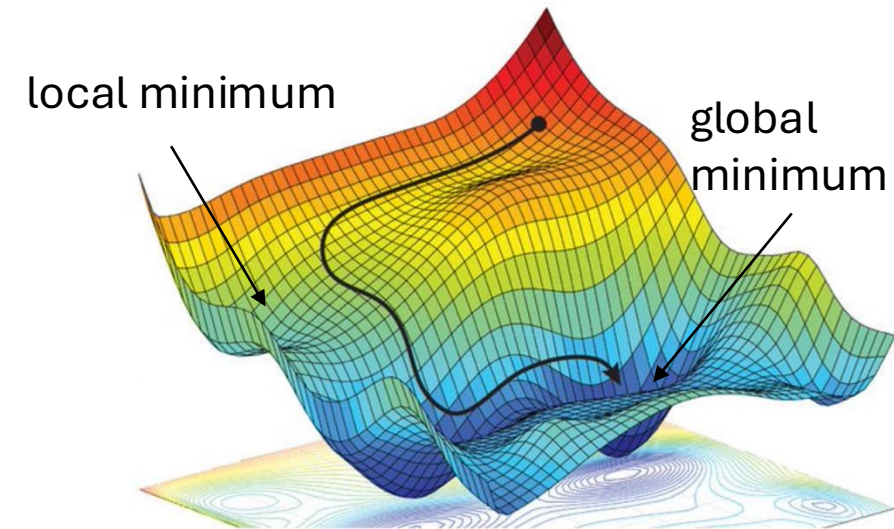
repeatedly take small steps downhill on the loss surface until a minimum is reached

typically use automatic differentiation (autodiff)
to calculate gradients (systematic application of
chain rule on computation graph)

called backpropagation in neural networks



[source](#)



Beyond Gradient Descent

gradient descent is simple and general, but not always the best choice

alternative optimization methods:

- second-order methods → faster convergence, more expensive
 - use curvature (Hessian)
 - Newton's method, quasi-Newton (BFGS)
 - stochastic methods (subsampling) → can escape shallow local minima, scales to large datasets, less stable
 - stochastic gradient descent (SGD)
 - mini-batch optimization
- } more on this later in deep learning discussion

→ appropriate optimization strategy depends on structure of $J(\boldsymbol{\theta})$

Generalization: Loss Functions

least squares: minimize sum of squared residuals

loss functions generalize squared residuals to arbitrary models and tasks: $L_i(\boldsymbol{\theta}) = L(y_i, f(\mathbf{x}_i; \boldsymbol{\theta}))$

$L_i(\boldsymbol{\theta})$ quantifies error (e.g., residual) on one sample

further examples:

- absolute error
- cross-entropy (classification)

Cost Function

aggregate (average) losses over data set:

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n L_i(\boldsymbol{\theta})$$

to be minimized according to model parameters $\boldsymbol{\theta}$

→ objective function

L^2 Regularization (Ridge Regression)

add **penalty term** on parameter L^2 norm to cost function

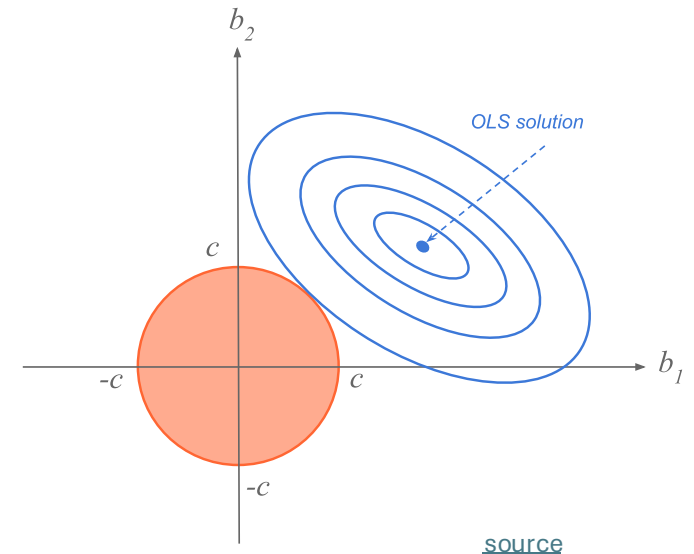
$$\tilde{J}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda \cdot \boldsymbol{\theta}^T \boldsymbol{\theta}$$

hyperparameter controlling
strength of regularization

aka **weight decay** in neural networks

corresponds to imposing constraint on parameters to lie in L^2 region (size controlled by λ)

goal: reduce overfitting from large/unstable parameters



Maximum Likelihood Estimation (MLE)

assume data is generated by $p(y|\mathbf{x}; \boldsymbol{\theta})$

idea: choose $\boldsymbol{\theta}$ to make observed data most likely

$$\hat{\boldsymbol{\theta}} = \operatorname{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^n p(y_i|\mathbf{x}_i; \boldsymbol{\theta}) = \operatorname{argmax}_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$$

likelihood function

independent and identically distributed (i.i.d.) samples

→ many loss functions defined from probabilistic assumptions

Side Note: i.i.d. Samples

identically distributed:

each sample drawn from the same distribution: $y_i \sim p(y|\mathbf{x}_i; \boldsymbol{\theta})$

→ same model, same parameters for all samples

independent:

→ additive losses: decompose over samples

samples do not influence each other: $p(y_1, \dots, y_n | X) = \prod_{i=1}^n p(y_i | \mathbf{x}_i)$

→ knowing one observation gives no information about another

common violations: time series (temporal dependence), spatial data (spatial dependence), text/language (sequential dependence)

Negative Log-Likelihood

often written as log
(means ln, but changing the base
only introduces a constant factor)

log-likelihood function

$$\ell(\boldsymbol{\theta}) = \ln(\mathcal{L}(\boldsymbol{\theta})) = \sum_{i=1}^n \ln(p(y_i | \mathbf{x}_i; \boldsymbol{\theta}))$$

- logarithm monotonic function $\rightarrow$ maximum of ℓ and $\mathcal{L}$ at same $\boldsymbol{\theta}$ values
- most probability distributions (e.g., exponential family) only logarithmically concave (important for optimization)
- sum computationally more convenient than product

negative log-likelihood: minimization instead of maximization

$$\hat{\boldsymbol{\theta}} = \operatorname{argmax}_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}) = \operatorname{argmin}_{\boldsymbol{\theta}} (-\ell(\boldsymbol{\theta}))$$

$\rightarrow$ negative log-likelihood: loss function derived from the likelihood

From Gaussian Noise to Least Squares

model assumption: $y_i = f(\mathbf{x}_i; \boldsymbol{\theta}) + \varepsilon_i$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$

implied distribution: $Y_i | X_i = \mathbf{x}_i \sim \mathcal{N}(f(\mathbf{x}_i; \boldsymbol{\theta}), \sigma^2)$

likelihood:

$$p(y_i | \mathbf{x}_i; \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2}{2\sigma^2}\right)$$

$$\ell(\boldsymbol{\theta}) = -\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2 + \text{const}$$

$$\hat{\boldsymbol{\theta}} = \operatorname{argmax}_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}) = \operatorname{argmin}_{\boldsymbol{\theta}} \sum_{i=1}^n (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2$$

→ least squares corresponds to MLE assuming Gaussian noise

different noise assumptions → different loss functions

(other examples: Bernoulli → cross-entropy, Laplace → absolute error)

Two Views on Uncertainty

frequentist perspective

- parameters θ fixed, but unknown
- data random: uncertainty comes from sampling variability
- estimate θ using data (likelihood) only (point estimates, potentially with confidence intervals)

Bayesian perspective

- parameters as random variables, following $p(\theta|y, \mathbf{x})$
- combine data (likelihood) and prior beliefs to express uncertainty via estimated posterior distributions

Bayes Theorem

H : hypothesis (model)

E : evidence (data)

likelihood function (derived
from statistical model)

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

posterior probability
(belief updated with
observed evidence)

new data (same
for all hypotheses
→ uninteresting)

prior probability
(belief in H
before evidence)

Bayes Theorem for Parameter Estimation

data likelihood:
model (function of θ with data fixed)

$$p(\theta|y, x) \propto p(y|x, \theta) \cdot p(\theta)$$

posterior:
probability distribution reflecting
the effect of data on prior belief
about parameters (increasing
certainty with new evidence)

to be compared to
frequentist point
estimates $\hat{\theta}$

prior (e.g., broad Gaussian):
probability distribution
reflecting initial belief about
uncertain parameters (helps to
reduce variance with few data)

→ Bayesian updating

Maximum A Posteriori (MAP) Estimation

MLE: mode of likelihood as point estimate

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} = \operatorname{argmax}_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \operatorname{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^n p(y_i | \mathbf{x}_i; \boldsymbol{\theta})$$

uses only the data, ignores prior knowledge

MAP: instead of only likelihood, include prior

$$\hat{\boldsymbol{\theta}}_{\text{MAP}} = \operatorname{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^n p(\boldsymbol{\theta} | y_i, \mathbf{x}_i) \underset{\substack{\uparrow \\ \text{Bayes}}}{=} \operatorname{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^n \underset{\substack{\downarrow \\ \text{likelihood} \\ \text{(MLE)}}}{p(y_i | \mathbf{x}_i; \boldsymbol{\theta})} \cdot \underset{\substack{\downarrow \\ \text{prior} \\ \text{(regularization)}}}{p(\boldsymbol{\theta})}$$

log: penalty term

- approximation to Bayesian inference (using mode of posterior distribution)
- MLE special case of MAP estimation with uniform prior

Gaussian Prior $\rightarrow$ Ridge Regression

$$\hat{\boldsymbol{\theta}}_{\text{MAP}} = \operatorname{argmin}_{\boldsymbol{\theta}} \left[-\sum_{i=1}^n \log(p(y_i | \mathbf{x}_i; \boldsymbol{\theta})) - \log p(\boldsymbol{\theta}) \right]$$

log of Gaussian prior $\boldsymbol{\theta} \sim \mathcal{N}(0, \tau^2 I)$:

$$\log p(\boldsymbol{\theta}) = -\frac{1}{2\tau^2} \|\boldsymbol{\theta}^2\| + \text{const}$$

prior belief:

- parameters centered around zero
- large values unlikely

log of Gaussian likelihood:

$$\log(p(y_i | \mathbf{x}_i; \boldsymbol{\theta})) = -\frac{1}{2\sigma^2} (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2 + \text{const}$$

$$\rightarrow \hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta}} \left[\sum_{i=1}^n (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2 + \underbrace{\frac{\sigma^2}{\tau^2} \|\boldsymbol{\theta}^2\|}_{\lambda} \right]$$

Priors as Regularizers

prior distribution: leveraging information that cannot be found in training data

MAP estimation explains regularization as incorporating prior knowledge into parameter estimation

many regularized estimation strategies can be interpreted as MAP

- L^2 regularization corresponds to MAP with Gaussian prior on parameters
- L^1 regularization corresponds to MAP with isotropic Laplace prior on parameters

MLE: data only

MAP: data + prior (regularization)

From Point Estimates to Uncertainty

two ways to express uncertainty

- Bayesian: credible intervals (probability about θ)
- frequentist: confidence intervals

go back to frequentist view: parameters fixed, uncertainty comes from data

so far point estimates for θ (single best value)

often need for certainty of these estimates

→ instead of a single value, calculate interval of plausible values

Confidence Intervals (CI)

95% CI: containing the true parameter in 95% of repeated experiments
expressed with α : $100(1 - \alpha)\%$

ingredients for CI:

1. estimator $\hat{\theta}$ (fixed)
2. its variability (standard error)
3. distribution (normal/asymptotic)

general form: $\hat{\theta} \pm (\text{critical value}) \times (\text{standard error})$

Standard Error (SE)

measures how much the estimate varies due to sampling:
If we repeated the experiment, how much would $\hat{\theta}$ fluctuate?

example: mean

$$SE(\mu) = \frac{s}{\sqrt{n}}$$

s : sample standard deviation

n : number of samples

CI depends on data: more data (repetitions of experiment) → smaller SE
→ more precise estimate (narrower interval)

Critical Value

probabilistic correction factor from Student's t-distribution:

$$t_{n-1, 1-\frac{\alpha}{2}}$$

- large $n \rightarrow t$ approaches standard normal
- small $n \rightarrow$ heavier tails $\rightarrow$ wider intervals

SE uses an estimate of the variance, but does not account for the uncertainty of that estimate itself

$\rightarrow$ t-distribution accounts for the extra uncertainty

Bootstrapping

idea: treat observed dataset as proxy for the true distribution and simulate new datasets from it

procedure: approximate the sampling distribution by resampling the data

- given data $x_1, \dots, x_n$: draw a bootstrap sample (with replacement)
- compute estimate $\hat{\theta}_b$
- repeat B times: $\hat{\theta}_1, \dots, \hat{\theta}_B$
- use empirical quantiles for CI: $[\text{quantile}_{\alpha/2}, \text{quantile}_{1-\alpha/2}]$